DocGuard

Frontend Build & Backend Integration Guide

A step-by-step implementation handoff for the frontend team

Project	DocGuard — Automated Documentation Drift Detection
Team	OdeX
Frontend owner	Frontend teammate / React implementation
Backend owner	Aditya — FastAPI, GitHub integration, analysis pipeline and Qwen/Ollama
Integration target	React + Vite frontend talking to FastAPI over HTTP/JSON
Document purpose	Build the UI independently while keeping API, data, runtime, and Git workflow compatible with the backend.

Core rule: the frontend displays backend results. It must not implement, simulate, or hardcode the documentation-drift analysis. The backend is the source of truth for findings, severity, evidence, explanations, and model status.

Prepared for the DocGuard hackathon MVP • Integration contract v1

1. Product goal and division of responsibility

DocGuard analyzes a GitHub pull request to find documentation that may have become incorrect after code changes. The frontend gives users a simple way to submit a repository and PR, monitor the request, and inspect evidence-backed findings.

Area	Frontend teammate owns	Backend owner owns
User interface	Dashboard, form, status, results, evidence presentation, responsive layout	No UI implementation required
GitHub	Collect repository URL and PR number; display submitted repository/PR details returned by API	Fetch PR metadata, changed files and diff; handle GitHub errors
Analysis	Render results exactly as returned; filter/sort findings locally if desired	Python AST/static facts, documentation mapping, drift reasoning
AI	Show model availability/status; display explanation/suggestion returned by API	Ollama + Qwen integration, prompts, validation and fallback/error handling
Integration	HTTP requests, loading/error/empty states, CORS-compatible configuration	FastAPI routes, validation, response schema, CORS configuration

Do not duplicate backend logic in JavaScript. Do not claim the model is running just because the frontend can load. Use the model-status endpoint.

2. Technology and project setup

Required: React, Vite, JavaScript (not TypeScript), npm, Tailwind CSS (or another consistent CSS approach), and Lucide React icons. Use Fetch or Axios for HTTP. Keep frontend code in a separate `frontend/` directory at the repository root.

Recommended frontend structure

```
DocGuard/
■■■ backend/                    # built by backend owner
■■■ frontend/                   # frontend teammate owns this folder
■   ■■■ public/
■   ■■■ src/
■   ■   ■■■ assets/
■   ■   ■■■ components/
■   ■   ■   ■■■ AppHeader.jsx
■   ■   ■   ■■■ RepoPrForm.jsx
■   ■   ■   ■■■ BackendStatus.jsx
■   ■   ■   ■■■ ModelStatus.jsx
■   ■   ■   ■■■ AnalysisSummary.jsx
■   ■   ■   ■■■ SeverityBreakdown.jsx
■   ■   ■   ■■■ FindingCard.jsx
■   ■   ■   ■■■ EvidencePanel.jsx
■   ■   ■   ■■■ LoadingState.jsx
■   ■   ■   ■■■ ErrorState.jsx
■   ■   ■■■ services/
■   ■   ■   ■■■ api.js
■   ■   ■■■ hooks/
■   ■   ■   ■■■ useAnalysis.js
■   ■   ■■■ utils/
■   ■   ■   ■■■ formatters.js
■   ■   ■■■ App.jsx
■   ■   ■■■ main.jsx
■   ■   ■■■ index.css
■   ■■■ .env.example
■   ■■■ .env.local              # local only; never commit
■   ■■■ package.json
■   ■■■ vite.config.js
■■■ ...
```

This is a suggested structure, not a requirement to create every component as a separate file. Keep components small and focused.

3. Required screens and user flow

The main user journey should be straightforward:

1. **Open dashboard:** show DocGuard name, one-line purpose, backend connectivity and model availability.
2. **Enter PR:** user pastes a public GitHub repository URL and enters a positive PR number.
3. **Validate input:** check URL format and PR number before sending. Do not assume that a syntactically valid URL means the repository or PR exists.
4. **Analyze:** disable duplicate submissions, show progress/loading feedback, and send one POST request to the backend.
5. **Display response:** show summary counts and all findings returned by the backend.
6. **Inspect evidence:** let the user open a finding and see the exact documentation sentence and corresponding code reference.
7. **Handle outcome:** show a clear completed-with-findings, completed-with-no-findings, or error state.

Minimum UI sections

Section	Required content
Header	DocGuard logo/name, short product description, optional GitHub repository link
Connection indicators	Backend reachable/unreachable; Qwen available/unavailable/unknown (from API)
PR form	Repository URL input, PR number input, Analyze PR button, validation messages
Summary	Analysis status, total finding count, high/medium/low counts
Findings list	Cards with type, severity, docs location, code location, confidence and short explanation
Evidence detail	Exact docs excerpt/value and code evidence; suggested correction
Footer / About	MVP scope note: supported file/language coverage is determined by backend implementation

Visual guidance

- Use a clean developer-tool dashboard. A dark theme is fine, but maintain readable contrast and accessible font sizes.
- Use severity colors consistently (for example: high = red, medium = amber, low = blue/neutral). Always include a text label; do not rely on color alone.
- Make the repository/PR form the primary action and keep it visible without excessive scrolling.
- Use clear code-style typography for file paths, line numbers, signatures and snippets.
- Design responsive layouts for laptop and smaller screens. Avoid layouts that break when explanations or paths are long.
- Do not show fake live progress percentages. Use an honest spinner/status message unless the backend provides real progress.

4. Backend API contract — version 1

Use the following paths and field names exactly. The backend owner will implement these routes. If a field or status changes, agree on the change before either side edits its implementation.

Method + path	Purpose	Frontend behavior
GET /health	Backend health check	Call on app load; show online/offline
GET /api/model-status	Ollama/Qwen availability	Show model available/unavailable/unknown

Method + path	Purpose	Frontend behavior
POST /api/analyze	Analyze a GitHub PR	Submit repo_url + pr_number; render returned JSON

4.1 GET /health — expected response

```
{
  "status": "ok",
  "service": "docguard-api"
}
```

Frontend rule: only mark the backend online if this request succeeds and returns a valid response. A browser/network error means offline or unreachable.

4.2 GET /api/model-status — expected response

```
{
  "available": true,
  "provider": "ollama",
  "model": "qwen3:4b",
  "message": "Model is ready"
}
```

The UI must support `available: false` and request failure. Do not block the whole dashboard if the model is unavailable; allow the user to see the status and show a useful message. The backend may still return an analysis error if AI is essential for that run.

4.3 POST /api/analyze — request body

```
{
  "repo_url": "https://github.com/owner/repository",
  "pr_number": 42
}
```

Validation expectations: repo_url is a public GitHub repository URL for the MVP; pr_number is a positive integer. The frontend can do basic validation, but backend validation remains authoritative.

5. Analysis response schema

The backend will return a JSON object using this agreed shape. The example is illustrative test data only; it is not a real analysis result and must not be hardcoded into the shipped UI.

```
{
  "status": "completed",
  "summary": {
    "total_findings": 1,
    "high": 1,
    "medium": 0,
    "low": 0
  },
  "findings": [
    {
      "id": "finding-001",
      "type": "api_reference",
      "severity": "high",
      "docs_file": "README.md",
      "docs_line": 24,
      "documented_value": "createUser(name, email)",
      "actual_value": "createUser(name, email, role='user')",
      "code_file": "src/users.py",
      "code_line": 10,
      "explanation": "The function now accepts an additional role parameter.",
      "suggestion": "Update the documentation to include the role parameter.",
      "confidence": 0.95
    }
  ]
}
```

Finding field definitions

Field	Type	How to display / handle
id	string	Stable key for React list rendering; do not generate a replacement if present
type	string	Finding category label; unknown values should still display safely
severity	string	Expected high, medium, low; handle unknown values with neutral styling
docs_file	string	Documentation path
docs_line	integer or null	Line number if known; show “line unavailable” if null
documented_value	string	Outdated text/value from documentation
actual_value	string	Actual value/signature/config fact found in code
code_file	string	Code path that supports the finding
code_line	integer or null	Code line if known; show “line unavailable” if null
explanation	string	Backend-generated explanation
suggestion	string	Suggested documentation correction
confidence	number	Expected 0–1; display as percentage only if valid

Important rendering rules

- Render the findings array dynamically. It may contain zero, one, or many findings.
- Do not assume a fixed finding type, order, or count.
- Treat missing optional values as unavailable rather than crashing.
- Escape/render backend strings as text. Do not inject them as raw HTML.
- Use the finding id as the React key when present; otherwise use a stable composite fallback.

- Show confidence as, for example, 95% only when the value is numeric and between 0 and 1. Do not label confidence as accuracy.
- Do not present a suggestion as an automatically applied code or documentation change.

6. Error handling and UI states

Situation	Expected frontend behavior
Initial / idle	Show empty state and ask for a repository URL + PR number
Submitting	Disable Analyze button; show “Analyzing pull request…”; prevent duplicate requests
Completed with findings	Show summary and findings; allow user to inspect each item
Completed with zero findings	Show “No documentation drift detected by this analysis”; clarify this does not prove docs are perfect
Backend unavailable	Show connection error and how to start/check backend
Model unavailable	Show model warning/status; explain AI-assisted analysis may not run
Invalid repository or PR	Show backend error message near the form; preserve entered values
GitHub rate limit / access issue	Show a clear message; do not silently claim analysis succeeded
Timeout / network failure	Show retry action; do not discard form values
Unexpected response	Show a generic safe error and log technical details in browser console during development

The backend may take longer than a typical small API request because it fetches a PR and analyzes changed files. Use a generous request timeout (for example, 120–180 seconds for the MVP) and show a loading state. Coordinate the final timeout with the backend owner. Do not implement polling unless the backend explicitly adds an asynchronous job API.

7. Frontend API client and environment

Create a single API client module (for example, `src/services/api.js`). Components should call this module rather than building fetch requests in multiple places.

Environment configuration

```
# frontend/.env.example
VITE_API_BASE_URL=http://localhost:8000
```

For local development, create `frontend/.env.local` with the same value. Vite exposes only variables prefixed with `VITE_` to browser code. Never put GitHub tokens, private keys, or secrets in a VITE_ variable.

Suggested API client behavior

```
const API_BASE = import.meta.env.VITE_API_BASE_URL;

export async function getHealth() {
  const response = await fetch(`${API_BASE}/health`);
  if (!response.ok) throw new Error("Backend health check failed");
  return response.json();
}

export async function getModelStatus() {
  const response = await fetch(`${API_BASE}/api/model-status`);
  if (!response.ok) throw new Error("Could not read model status");
  return response.json();
}

export async function analyzePullRequest(payload) {
  const response = await fetch(`${API_BASE}/api/analyze`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload)
  });

  const data = await response.json().catch(() => ({}));
  if (!response.ok) {
    throw new Error(data.detail || data.message || "Analysis failed");
  }
  return data;
}
```

The backend may return validation details in a different error format; make the client robust if detail/message is missing. Never assume every error response is JSON.

8. CORS and local network integration

For development on one Mac, frontend runs at `http://localhost:5173` and backend at `http://localhost:8000`. These are different origins, so the backend must allow the frontend origin through CORS.

Backend owner will configure CORS for the exact development origins, typically localhost and 127.0.0.1 on port 5173. The frontend teammate should not try to solve CORS by disabling browser security or using a random proxy.

When teammates access the frontend from their own laptops on the same Wi-Fi:

- Backend must listen on the Mac's LAN interface (for example, host 0.0.0.0 on port 8000).
- Frontend must use the Mac's LAN IP in VITE_API_BASE_URL, for example http://192.168.1.25:8000. The actual IP will be supplied by the backend owner.
- Backend CORS must allow the frontend origin used by the teammate's browser. For Vite dev server, use host 0.0.0.0 if other devices must open it.
- Mac must remain awake, connected to the same network, and running both backend and Ollama.
- Ollama should remain bound to localhost. The frontend must never call Ollama directly; only the backend calls localhost:11434.
- Do not expose the unauthenticated Ollama port to the LAN or public internet.

Network example

```
Teammate's browser
  -> http://<MAC-LAN-IP>:5173  (Vite frontend)
  -> http://<MAC-LAN-IP>:8000  (FastAPI API)
  -> http://localhost:11434   (Ollama, on the Mac only)
```

For the final demo, both frontend and backend must be configured to use the same reachable host. localhost on a teammate's laptop refers to that teammate's laptop, not the backend owner's Mac.

9. Frontend build steps — in order

1. **Initialize project:** from repository root, create a Vite React JavaScript project inside frontend/. Do not select the TypeScript template.
2. **Install dependencies:** install the UI libraries the team chooses (Tailwind CSS, Lucide React, and optionally Axios). Keep package.json and package-lock.json committed.
3. **Create layout:** build header, main content area, PR form, status indicators, results section and responsive styling.
4. **Create API client:** add api.js with health, model-status and analyze methods. Read base URL from VITE_API_BASE_URL.
5. **Connect health:** call GET /health on initial load and show online/offline state. Avoid rapid repeated polling.
6. **Connect model status:** call GET /api/model-status and display the returned provider/model/message. Handle unavailable and network failure.
7. **Build PR form:** collect repo_url and pr_number; validate basic input; submit exact JSON request.
8. **Build loading and error states:** disable repeated submit, show spinner/message, preserve input after errors, and allow retry.
9. **Build result summary:** use summary.total_findings and summary.high/medium/low from response. Do not recalculate or hardcode counts.
10. **Build findings list:** map over findings and display every finding field with safe fallbacks.
11. **Build evidence detail:** show docs path/line and code path/line together; display documented_value vs actual_value, explanation and suggestion.
12. **Test edge cases:** zero findings, many findings, missing line number, unknown severity/type, model unavailable, backend down, invalid PR and slow response.
13. **Commit frontend:** work on a feature branch and open a PR; do not push directly to main if the hackathon repository rules require PR-only changes.

Suggested commands

```
# Run from DocGuard repository root
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install lucide-react
# Install/configure Tailwind using the current Tailwind + Vite instructions.
npm run dev
```

The backend teammate will separately configure FastAPI and CORS. Do not invent backend endpoints or change the API contract without discussing it.

10. Testing before integration

Test	Expected result
Backend health reachable	Green/online indicator
Model status available	Provider and model displayed from response
Valid public repo + PR	Loading state followed by backend response
Completed response with one finding	One finding card and correct summary counts
Completed response with empty findings	Friendly zero-findings state
Backend stopped	Clear offline/error message; no blank screen
Model stopped/unavailable	Model warning; no false “AI ready” claim

Test	Expected result
Invalid PR number	Client validation or backend validation message
Slow API response	Loading remains visible; no duplicate requests
Long file paths / text	Layout wraps without overflow
Mobile/narrow viewport	Form and findings remain usable

If the backend is not ready yet, the frontend teammate may temporarily use a local mock JSON file to build the layout. Mark it clearly as development-only, keep it separate from production API code, and remove or disable it before the final demo. The final Analyze action must call the real backend.

11. Git workflow and collaboration

- Keep frontend changes inside `frontend/` unless a shared root configuration is explicitly agreed.
- Create a dedicated branch such as `feat/frontend-dashboard`.
- Do not commit `node_modules/`, `dist/`, `.env.local`, API tokens or secrets.
- Commit `package.json` and `package-lock.json` so the project can be installed reproducibly.
- Open a PR and describe UI features, API endpoints connected, test steps, and any assumptions.
- Coordinate shared-file edits (README, root .gitignore, PR template, workflows) with the backend/repo owner.
- Do not push directly to main if the hackathon's repository rules prohibit it.

12. Run instructions for the demo

Backend owner starts the backend and Ollama on their Mac. Frontend teammate starts Vite. Exact commands may be adjusted by the backend owner as the implementation is completed.

```
# Terminal 1 — backend owner
cd DocGuard/backend
source venv/bin/activate
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# Terminal 2 — backend owner (if Ollama app/service is not already running)
ollama serve

# Terminal 3 — frontend
cd DocGuard/frontend
npm install
npm run dev
```

For same-Mac development, frontend .env.local uses http://localhost:8000. For another device on the same Wi-Fi, replace localhost with the backend Mac's LAN IP. Confirm the Mac firewall allows the required development ports.

13. Definition of done — frontend

- React/Vite app starts with documented commands.
- All UI is built in JavaScript; no TypeScript.
- User can enter a GitHub repo URL and PR number.
- Health and model status are fetched from real backend endpoints.
- Analyze PR calls POST /api/analyze with the exact request schema.
- Loading, success, zero-finding, validation, network and backend error states work.
- Every returned finding is rendered dynamically with docs/code evidence, explanation, suggestion and confidence.
- No fake findings, fake model status, fake progress percentages or hardcoded summary counts remain in the final path.
- Frontend base URL is configurable through .env.local and .env.example is committed.
- Works on the same Mac and can be configured for same-Wi-Fi demo access.
- README includes setup/run steps; no secrets or local environment files are committed.
- Frontend branch/PR is ready for integration.

14. Questions to coordinate with the backend owner

Confirm before final integration	Owner / current decision
API paths and JSON field names	Backend owner publishes this contract; discuss before changing

Confirm before final integration	Owner / current decision
CORS allowed origins	Backend owner configures; frontend reports actual Vite origin
Final LAN IP and port	Backend owner supplies for multi-device demo
Request timeout	Agree together; initial suggestion 120–180 seconds
Error response shape	Backend owner shares actual examples once implemented
Supported languages/file types	Backend owner documents actual MVP support; frontend must not overclaim
Whether analysis can partially succeed	Backend owner defines final status semantics
Final model name and readiness message	Backend owner reports actual Ollama/Qwen state

15. Quick handoff checklist

The frontend teammate should work through this checklist in order:

- [ ] Create frontend/ using Vite React JavaScript template.
- [ ] Install dependencies and commit package files.
- [ ] Build dashboard shell and responsive layout.
- [ ] Build repository URL + PR number form.
- [ ] Add .env.example and local VITE_API_BASE_URL configuration.
- [ ] Implement centralized API client.
- [ ] Connect GET /health.
- [ ] Connect GET /api/model-status.
- [ ] Connect POST /api/analyze.
- [ ] Implement loading, validation, error and retry states.
- [ ] Implement summary counts from backend response.
- [ ] Implement dynamic finding cards.
- [ ] Implement evidence comparison view.
- [ ] Test zero, one and multiple findings.
- [ ] Test unavailable backend/model and slow requests.
- [ ] Verify no secrets, .env.local, node_modules or dist are committed.
- [ ] Open frontend PR and share branch/PR with backend owner.

16. Sample response for UI development only

If the backend endpoint is not ready, the frontend teammate may save the following as a temporary fixture to design the result cards. It must not be used as a real result or remain wired to the final Analyze button.


```
{
  "status": "completed",
  "summary": {
    "total_findings": 2,
    "high": 1,
    "medium": 1,
    "low": 0
  },
  "findings": [
    {
      "id": "demo-001",
      "type": "api_reference",
      "severity": "high",
      "docs_file": "README.md",
      "docs_line": 24,
      "documented_value": "createUser(name, email)",
      "actual_value": "createUser(name, email, role='user')",
      "code_file": "src/users.py",
      "code_line": 10,
      "explanation": "Demo only: the documented signature omits a parameter.",
      "suggestion": "Demo only: document the role parameter and its default.",
      "confidence": 0.95
    },
    {
      "id": "demo-002",
      "type": "configuration_default",
      "severity": "medium",
      "docs_file": "docs/configuration.md",
      "docs_line": 12,
      "documented_value": "timeout = 10",
      "actual_value": "timeout = 30",
      "code_file": "app/config.py",
```

```
      "code_line": 8,
      "explanation": "Demo only: the documented default differs from code.",
      "suggestion": "Demo only: update the documented default to 30.",
      "confidence": 0.88
    }
  ]
}
```

17. Final integration principle

One source of truth: backend analysis determines what is wrong and why; frontend determines how to present it. If a result is absent from the API response, the frontend must not invent it. If the backend contract needs to change, communicate the change and update both sides together.

End of guide • DocGuard / OdeX